

Global AI Olympiad: Comprehensive Website Structure and User Experience Strategy

Project Name: Britsync **Global AI Tech Olympiad**

Prepared By: Sumaiya

Event Summary

The Global AI Olympiad is a virtual competition aimed at fostering innovation and collaboration in local community development through AI solutions. Teams will compete to create impactful AI projects that address real-world problems.

Event Planning for the Global AI Olympiad

1. Event Overview:

- **Organizer:** Britsync, an AI-based digital marketing solution provider.
- **Theme:** "AI for Local Community Development."
- **Format:** Online competition, with no physical final event (optional gala event planned at Royal Museum of Science, London).
- **Target Teams:** 100 teams consisting of finance, tech, branding, and marketing professionals.
- **Duration:** 6 weeks
- Stages: Preparation, team formation, training, project development, and final submission.

2. Event Structure:

- **Pre-Event Planning:**
 - Define key objectives and evaluation criteria for submissions.
 - Develop event branding, logo, and promotional materials.
 - Build a website and create a social media marketing strategy.
- **Registration Process:**
 - Open online registration for teams.
 - Promote the event via email outreach (targeting universities).
- **Workshops and Training:**
 - Pre-event online workshops on AI tools, project management, and branding.
 - Provide mentorship from industry experts.
- **Hackathon and Project Development:**
 - Conduct a hackathon where teams develop AI-driven solutions.
 - Schedule regular mentor check-ins.
- **Submission and Evaluation:**
 - Define submission deadlines and formats (video pitch, documentation).
 - Evaluate projects based on innovation, impact, and feasibility.

- **Awards and Recognition:**
 - Categories for prizes include Best Innovation, Most Impactful Solution, and Audience Choice.
 - Incentives such as cash prizes, mentorship opportunities, and certificates.
-

Event Structure:

1. Pre-Event Planning (Weeks 1-2)

A. Objectives

- Identify key goals (e.g., number of teams, project types).
- Define evaluation criteria for project submissions.

B. Branding and Marketing

- Develop event branding, including logo, tagline, and promotional materials.
- Create a dedicated website/landing page for information and registration.
- Develop a social media strategy for promotions on platforms like LinkedIn, Twitter, and Instagram.

C. Outreach and Partnerships

- Collaborate with universities, tech companies, and NGOs to promote the event and recruit teams.
- Reach out to potential sponsors for prizes and mentorship (e.g., AI firms, tech giants).

2. Registration and Team Formation (Week 3)

A. Registration Process

- Open online registration for teams through the event website.
- Create an application form to collect team details, including names, skills, and areas of interest.

B. Team Composition

- Encourage teams to have a mix of finance, tech, and branding marketing members.
- Provide resources (e.g., a forum or platform) for individuals to connect and form teams.

C. Promotion

- Announce registration via social media, email newsletters, and press releases.

3. Event Kickoff (Week 4)

A. Opening Ceremony

- Host a virtual opening ceremony with keynote speakers from the AI and community development sectors.
- Introduce the event format, timeline, rules, and evaluation criteria.

B. Workshops and Training Sessions

- Schedule workshops on relevant topics, such as:
 - AI technologies and tools
 - Project management
 - Effective branding and marketing strategies
 - Financial planning for projects
- Ensure sessions are recorded for later access.

4. Competition Phase (Weeks 5-6)

A. Project Development

- Teams work on their AI-driven projects focused on local community development.
- Provide resources and access to AI tools, datasets, and mentorship throughout this phase.

B. Progress Checkpoints

- Schedule bi-weekly check-ins for teams to share updates and receive feedback from mentors.
- Use a collaboration platform (e.g., Slack, Discord) for teams to communicate and ask questions.

5. Submission and Evaluation (End of Week 6)

A. Project Submission

- Define submission guidelines and deadlines for final project presentations (e.g., video pitch, documentation).
- Create a submission portal on the event website for easy uploads.

B. Evaluation Process

- Assemble a panel of judges consisting of industry experts in AI, finance, and marketing.
- Evaluate projects based on:
 - Innovation
 - Impact
 - Feasibility

- Presentation quality

6. Awards and Recognition (Week 7)

A. Announcement of Winners

- Host a virtual awards ceremony to announce winners and showcase standout projects.
- Consider live streaming the ceremony on platforms like YouTube or Facebook.

B. Prizes and Incentives

- Provide prizes for top teams (e.g., cash prizes, mentorship opportunities, software licenses).
- Offer certificates of participation for all teams to recognize their efforts.

C. Post-Event Support

- Create a post-event community for participants to continue networking and sharing progress.
- Consider establishing a fund or incubator program to support the implementation of promising projects.

7. Post-Event Activities (Ongoing)

A. Follow-up and Impact Assessment

- Gather feedback from participants to improve future events.
- Assess the impact of the projects developed during the Olympiad.

B. Continued Engagement

- Share success stories through newsletters and social media.
- Explore opportunities for ongoing collaboration with participants and partners.

Budget Considerations

- **Marketing Materials:** Design and production costs for promotional content.
- **Platform Subscription Fees:** Costs for hosting workshops and events online.
- **Prizes:** Financial incentives for winners.
- **Web Development:** For event website and submission portal.

- **Speakers and Mentors:** Honoraria for keynote speakers and mentors.

Conclusion

This structured event plan aims to create an engaging and impactful Global AI Olympiad that brings together diverse teams to harness AI for local community development. By focusing on finance, tech, and branding marketing, Britsync can foster innovation and drive positive change globally.

Website Structure and Details for the Global AI Olympiad

1. Homepage

- **Header:**
 - Event Logo
 - Navigation Menu (Home, About, Registration, Workshops, Submission, Prizes, FAQs, Contact)
- **Hero Section:**
 - Engaging banner image related to AI and community development.
 - Tagline: "Harnessing AI for Local Community Development"
 - Call to Action (CTA) button: "Register Now"
- **Overview:**
 - Brief introduction to the event, including objectives and themes.
- **Countdown Timer:**
 - Countdown to the registration deadline and event kickoff.

2. About Page

- **Event Mission:**
 - Detailed description of the event's goals and vision.
- **Organizer Information:**
 - Overview of Britsync, its mission, and previous initiatives.
- **Theme Explanation:**
 - Discuss the importance of AI for local community development.

3. Registration Page

- **Team Registration Form:**
 - Fields for team name, member details (name, role, email, expertise), and project ideas.
 - Checkboxes for skills (Finance, Tech, Branding/Marketing).
- **Registration Guidelines:**
 - Instructions on how to form a team and register.
 - Information on deadlines and requirements.

- **Call to Action:**
 - Button to "Submit Registration"

4. Workshops Page

- **Workshop Schedule:**
 - List of workshops with dates, times, and topics (AI technologies, project management, branding/marketing).
- **Registration Links:**
 - Sign-up forms for each workshop (if needed).
- **Resources:**
 - Links to recorded sessions for later access.

5. Submission Page

- **Project Submission Guidelines:**
 - Detailed instructions on how to submit projects (formats, deadlines).
- **Submission Portal:**
 - Link to the submission portal (could be a form or document upload feature).
- **Evaluation Criteria:**
 - Outline the criteria by which projects will be judged.

6. Prizes Page

- **Awards Overview:**
 - List of prizes for winners (e.g., cash prizes, mentorship opportunities).
- **Judging Panel:**
 - Introduction to judges, their backgrounds, and expertise.
- **Recognition:**
 - Information on certificates of participation for all teams.

7. FAQs Page

- **Common Questions:**
 - Address typical inquiries about registration, team formation, project requirements, and event logistics.
- **Contact Information:**
 - Provide an email address for additional questions.

8. Contact Page

- **Contact Form:**
 - Fields for name, email, subject, and message.
- **Social Media Links:**

- Icons linking to Britsync's social media profiles (LinkedIn, Twitter, Facebook).
- **Newsletter Signup:**
 - Option for visitors to subscribe for updates about the event.

9. Blog/News Section (Optional)

- **Updates and Announcements:**
 - Regular posts about event updates, participant highlights, and relevant articles on AI and community development.

10. Footer

- **Navigation Links:** Quick links to all main sections.
- **Copyright Information:** Year and event name.
- **Privacy Policy:** Link to the privacy policy detailing how participant data will be used.

Additional Considerations

- **User Experience:** Ensure the website is mobile-friendly and easy to navigate.
- **SEO Optimization:** Use relevant keywords to improve search visibility.
- **Accessibility:** Follow web accessibility guidelines to ensure the site is usable for all individuals.

Hosting and Domain

- **Domain Name:** Consider using a straightforward domain like **www.globalaiolympiad.com** or **www.britsync-ai-olympiad.com**.
- **Web Hosting:** Choose a reliable hosting provider (e.g., Bluehost, SiteGround, or GoDaddy) that supports scalability and security.

This website structure will provide a comprehensive and user-friendly platform for the Global AI Olympiad, ensuring that participants have all the information they need to engage effectively in the competition.

Implement various types of bots

To streamline the process of organizing the **Global AI Olympiad** and enhance participant engagement, you can implement various types of bots and automation tools. Here are some options that can effectively manage different aspects of the event:

1. Event Management Bot

- **Functionality:** Automate event registration, ticketing, and participant communications. This bot can handle inquiries about the event, provide information about deadlines, and send reminders for upcoming workshops and deadlines.
- **Platform:** Consider using chatbots on platforms like **Slack**, **Discord**, or your event website.

2. Mentorship Coordination Bot

- **Functionality:** Match participants with mentors based on their skills and needs. The bot can schedule mentorship sessions, send reminders, and collect feedback on sessions to improve the mentorship experience.
- **Platform:** Integrate with tools like **Calendly** or **Airtable** for scheduling and tracking.

3. Project Submission and Review Bot

- **Functionality:** Manage project submissions, track deadlines, and facilitate the judging process. This bot can collect submissions, send confirmation emails, and notify teams about the judging timeline and results.
- **Platform:** Use platforms like **Google Forms** or **Typeform** integrated with a project management tool like **Trello** or **Asana**.

4. Voting and Feedback Bot

- **Functionality:** Facilitate community voting for the "Fan Favorite" award and collect feedback from participants after the event. The bot can manage the voting process securely and provide results to the organizing committee.
- **Platform:** Use polling tools like **Poll Everywhere**, **Slido**, or custom-built forms.

5. Communication and Engagement Bot

- **Functionality:** Maintain engagement with participants through regular updates, reminders, and important announcements. This bot can also facilitate discussions and encourage networking among participants.
- **Platform:** Integrate with messaging platforms like **WhatsApp**, **Telegram**, or email marketing tools like **Mailchimp**.

6. Gala Event Coordination Bot

- **Functionality:** Specifically for the final gala event at the Royal Museum of Science, this bot can handle RSVP management, seating arrangements, and event schedules. It can also provide attendees with event details, such as location, timings, and dress codes.
- **Platform:** Use an event management tool like **Eventbrite** or **Cvent** to integrate a bot that assists with all logistical aspects of the gala.

7. Feedback and Evaluation Bot

- **Functionality:** After the event, this bot can send out surveys to participants, judges, and mentors to collect feedback on their experiences and suggestions for improvement.
- **Platform:** Use tools like **SurveyMonkey** or **Google Forms** for this purpose, and integrate with your communication channels.

Implementation Strategy

1. **Choose a Central Platform:** Select a primary platform (e.g., Slack, Discord, or a dedicated event website) where most of the bots will be deployed to facilitate seamless communication.
2. **Integrate Bots:** Use APIs and integration tools like **Zapier** or **Integromat** to connect various bots and tools to streamline workflows.
3. **Test Functionality:** Before the event, conduct thorough testing of all bots to ensure they work properly and address any potential issues.
4. **Train Participants:** Provide guides or training sessions on how to use these bots, ensuring participants can easily access information and engage with the features.

Creating the chatbots for our Discord server

Below is a step-by-step high-level guide to creating the seven chatbots for your **Discord** server, along with code examples for each bot. This guide assumes you have some familiarity with coding in **JavaScript** and using **Node.js**. Make sure you have **Discord.js** library installed for interacting with the Discord API.

Prerequisites

1. **Node.js** installed on your system.
2. A **Discord account** and a server where you have permission to add bots.
3. **Discord bot token:** Create a bot on the Discord Developer Portal and get the token.

Step 1: Setting Up Your Project

1. Create a new directory for your project:

```
bash

mkdir discord-bots
cd discord-bots
```

2. Initialize the project:

```
bash

npm init -y
```

3. Install the necessary packages:

```
bash

npm install discord.js axios dotenv
```

4. Create a `.env` file to store your Discord bot token:

```
makefile

DISCORD_TOKEN=your_bot_token_here
```

5. Create the main bot file (e.g., `index.js`):

```
javascript

const { Client, GatewayIntentBits } = require('discord.js');
const dotenv = require('dotenv');

dotenv.config();

const client = new Client({ intents: [GatewayIntentBits.Guilds,
GatewayIntentBits.GuildMessages, GatewayIntentBits.MessageContent] });

client.once('ready', () => {
  console.log(`Logged in as ${client.user.tag}`);
});

client.login(process.env.DISCORD_TOKEN);
```

Step 2: Creating Each Bot

Here's a high-level guide and example code for each of the seven chatbots you want to create.

1. Event Management Bot

This bot manages event registration and participant communications.

Code Example:

```
javascript

// Add this to your index.js

client.on('messageCreate', (message) => {
  if (message.content.startsWith('!register')) {
    // Parse command arguments
    const args = message.content.split(' ').slice(1);
    const participantName = args.join(' ');

    // Store participant registration (you might want to store this in a
    database)
    message.channel.send(`${participantName} has registered for the
    event!`);
  }
});
```

2. Mentorship Coordination Bot

This bot matches participants with mentors and schedules sessions.

Code Example:

```
javascript

// Add this to your index.js

client.on('messageCreate', (message) => {
  if (message.content.startsWith('!mentor')) {
    // You can implement logic to connect participants with mentors here
    message.channel.send('Please wait while we find a mentor for you...');
    // Logic to match mentors and participants
  }
});
```

3. Project Submission and Review Bot

This bot manages project submissions and provides feedback.

Code Example:

```
javascript

// Add this to your index.js
```

```

client.on('messageCreate', (message) => {
  if (message.content.startsWith('!submit')) {
    const projectDetails = message.content.split(' ').slice(1).join(' ');
    // Store the project submission (you might want to store this in a
    database)
    message.channel.send(`Project submitted: ${projectDetails}`);
  }
});

```

4. Voting and Feedback Bot

This bot facilitates community voting for the "Fan Favorite" award.

Code Example:

```

javascript

// Add this to your index.js

client.on('messageCreate', (message) => {
  if (message.content.startsWith('!vote')) {
    const voteOption = message.content.split(' ').slice(1).join(' ');
    // Store vote (you might want to store this in a database)
    message.channel.send(`Vote recorded for: ${voteOption}`);
  }
});

```

5. Communication and Engagement Bot

This bot sends reminders and important announcements.

Code Example:

```

javascript

// Add this to your index.js

client.on('messageCreate', (message) => {
  if (message.content.startsWith('!announce')) {
    const announcement = message.content.split(' ').slice(1).join(' ');
    message.channel.send(`Announcement: ${announcement}`);
  }
});

```

6. Gala Event Coordination Bot

This bot handles RSVP management for the final gala event.

Code Example:

```

javascript

```

```
// Add this to your index.js

client.on('messageCreate', (message) => {
  if (message.content.startsWith('!rsvp')) {
    const name = message.content.split(' ').slice(1).join(' ');
    message.channel.send(`${name} has RSVP'd for the gala event!`);
  }
});
```

7. Feedback and Evaluation Bot

This bot collects feedback from participants after the event.

Code Example:

```
javascript

// Add this to your index.js

client.on('messageCreate', (message) => {
  if (message.content.startsWith('!feedback')) {
    const feedback = message.content.split(' ').slice(1).join(' ');
    message.channel.send(`Feedback received: ${feedback}`);
  }
});
```

Step 3: Running Your Bot

1. Start your bot:

```
bash

node index.js
```

2. Interact with your bot in Discord using the commands you've set up (e.g., !register, !mentor, etc.).

Step 4: Enhancements and Persistence

To improve functionality:

- **Database Integration:** Use a database like MongoDB or SQLite to store registrations, project submissions, votes, and feedback persistently.
- **Command Handling:** Consider implementing a command handler for better organization as your bot grows.
- **Advanced Features:** Implement scheduled messages for reminders using setTimeout or libraries like node-cron.

Additional Notes

- Make sure to manage permissions for the bot in your Discord server to prevent unauthorized actions.
- Test each feature thoroughly in a controlled environment before going live.
- Continuously iterate on the bot functionalities based on participant feedback.

This framework provides a solid foundation for your event management through Discord, allowing your tech team to develop and expand these functionalities based on the specific needs of the Global AI Olympiad.

To create a refined and scalable system for your **Global AI Olympiad** using Discord bots, we can enhance the original design to include file training capabilities, centralized analytics, and API optimizations to manage expenses effectively. Below, I outline a detailed approach incorporating your preferences and suggestions.

High-Level Overview

1. **Discord Bot Framework:** A modular bot structure that supports different functionalities.
2. **File Handling:** Allow bots to read and train on files, PDFs, links, etc.
3. **Centralized Analytics Dashboard:** Track engagement, submissions, votes, and feedback.
4. **API Management:** Use API limiters and optimizers to minimize costs.

Prerequisites

1. **Node.js** installed on your system.
2. **Discord.js** for bot integration.
3. **Database** (e.g., MongoDB, PostgreSQL) for persistent data storage.
4. **API Services** for file processing, data analytics, etc.
5. **Libraries:**
 - `pdf-parse` for PDF handling.
 - `axios` for API requests.
 - `express` for creating an analytics dashboard.

Step 1: Setting Up Your Project

1. **Create a new directory for your project:**

```
bash

mkdir global-ai-olympiad-bots
cd global-ai-olympiad-bots
```


2. Initialize the project:

```
bash

npm init -y
```

3. Install the necessary packages:

```
bash

npm install discord.js axios dotenv pdf-parse mongoose express rate-limit-axios
```

4. Create a .env file to store your Discord bot token and database connection string:

```
plaintext

DISCORD_TOKEN=your_bot_token_here
DATABASE_URL=your_database_connection_string_here
```

5. Create the main bot file (e.g., index.js):

```
javascript

const { Client, GatewayIntentBits } = require('discord.js');
const dotenv = require('dotenv');
const mongoose = require('mongoose');

dotenv.config();

const client = new Client({ intents: [GatewayIntentBits.Guilds,
GatewayIntentBits.GuildMessages, GatewayIntentBits.MessageContent] });

mongoose.connect(process.env.DATABASE_URL, { useNewUrlParser: true,
useUnifiedTopology: true })
  .then(() => console.log('Connected to the database'))
  .catch(err => console.error('Database connection error:', err));

client.once('ready', () => {
  console.log(`Logged in as ${client.user.tag}`);
});

client.login(process.env.DISCORD_TOKEN);
```

Step 2: Creating Each Bot with Enhanced Features

1. Event Management Bot

This bot manages event registration, allows file uploads for training, and communicates with participants.

Code Example:

```
javascript

const { MessageAttachment } = require('discord.js');

// In your index.js, add the following under your bot setup:

client.on('messageCreate', async (message) => {
  if (message.content.startsWith('!register')) {
    const args = message.content.split(' ').slice(1);
    const participantName = args.join(' ');

    // Save to the database
    const newParticipant = new Participant({ name: participantName });
    await newParticipant.save();

    message.channel.send(`${participantName} has registered for the event!`);

    // Check for file uploads
    if (message.attachments.size > 0) {
      const attachment = message.attachments.first();
      // Process the file (e.g., PDF training)
      if (attachment.name.endsWith('.pdf')) {
        const fileData = await processPDF(attachment.url);
        // Logic to train AI models based on PDF data
        message.channel.send(`File ${attachment.name} processed for training.`);
      }
    }
  }
});

async function processPDF(url) {
  const axios = require('axios');
  const pdfParse = require('pdf-parse');

  const response = await axios.get(url, { responseType: 'arraybuffer' });
  const dataBuffer = Buffer.from(response.data);

  return pdfParse(dataBuffer);
}
```

2. Mentorship Coordination Bot

This bot matches participants with mentors and allows file sharing for mentoring.

Code Example:

```
javascript

client.on('messageCreate', (message) => {
  if (message.content.startsWith('!mentor')) {
```

```

        message.channel.send('Please wait while we find a mentor for you...');
        // Logic to match mentors
    }

    // Allow mentors to share files/resources
    if (message.attachments.size > 0) {
        const attachment = message.attachments.first();
        // Store attachment URL for later reference
        message.channel.send(`Resource shared by mentor: ${attachment.url}`);
    }
});

```

3. Project Submission and Review Bot

This bot handles project submissions, file uploads, and provides feedback.

Code Example:

```

javascript

client.on('messageCreate', async (message) => {
    if (message.content.startsWith('!submit')) {
        const projectDetails = message.content.split(' ').slice(1).join(' ');
        const newSubmission = new Submission({ details: projectDetails, userId:
message.author.id });
        await newSubmission.save();

        message.channel.send(`Project submitted: ${projectDetails}`);

        // Handle file uploads
        if (message.attachments.size > 0) {
            const attachment = message.attachments.first();
            if (attachment.name.endsWith('.pdf')) {
                const fileData = await processPDF(attachment.url);
                // Store processed file data for evaluation
                message.channel.send(`File ${attachment.name} submitted for
review.`);
            }
        }
    }
});

```

4. Voting and Feedback Bot

This bot facilitates community voting and collects feedback via file uploads.

Code Example:

```

javascript

```

```

client.on('messageCreate', (message) => {
  if (message.content.startsWith('!vote')) {
    const voteOption = message.content.split(' ').slice(1).join(' ');
    const newVote = new Vote({ option: voteOption, userId:
message.author.id });
    await newVote.save();

    message.channel.send(`Vote recorded for: ${voteOption}`);
  }

  // Collect feedback
  if (message.content.startsWith('!feedback')) {
    const feedback = message.content.split(' ').slice(1).join(' ');
    const newFeedback = new Feedback({ content: feedback, userId:
message.author.id });
    await newFeedback.save();

    message.channel.send(`Feedback received: ${feedback}`);
  }
});

```

5. Communication and Engagement Bot

This bot sends reminders and important announcements, and allows for scheduled messages.

Code Example:

```

javascript

client.on('messageCreate', (message) => {
  if (message.content.startsWith('!announce')) {
    const announcement = message.content.split(' ').slice(1).join(' ');
    message.channel.send(`Announcement: ${announcement}`);

    // Optionally schedule announcements
    // Use node-cron or similar to schedule messages
  }
});

```

6. Gala Event Coordination Bot

This bot handles RSVP management and can include file uploads for dietary preferences.

Code Example:

```

javascript

client.on('messageCreate', async (message) => {
  if (message.content.startsWith('!rsvp')) {

```

```

const name = message.content.split(' ').slice(1).join(' ');
const newRsvp = new RSVP({ name: name, userId: message.author.id });
await newRsvp.save();

message.channel.send(`${name} has RSVP'd for the gala event!`);

// Handle dietary preference files
if (message.attachments.size > 0) {
  const attachment = message.attachments.first();
  message.channel.send(`Dietary preference shared:
  ${attachment.url}`);
}
});

```

7. Feedback and Evaluation Bot

This bot collects feedback from participants and uses files for more detailed insights.

Code Example:

```

javascript

client.on('messageCreate', (message) => {
  if (message.content.startsWith('!feedback')) {
    const feedback = message.content.split(' ').slice(1).join(' ');
    const newFeedback = new Feedback({ content: feedback, userId:
message.author.id });
    await newFeedback.save();

    message.channel.send(`Feedback received: ${feedback}`);

    // Optionally handle file uploads for detailed feedback
    if (message.attachments.size > 0) {
      const attachment = message.attachments.first();
      message.channel.send(`Additional feedback file shared:
      ${attachment.url}`);
    }
  }
});

```

Step 3: Setting Up Centralized Analytics Dashboard

1. Create an Express Server for the analytics dashboard:

```

javascript

const express = require('express');
const app = express();
const PORT = 3000;

app.use(express.json());

```

```

app.get('/analytics', async (req, res) => {
  // Fetch and process data from your database
  const submissions = await Submission.find();
  const feedbacks = await Feedback.find();
  const votes = await Vote.find();

  // Return analytics data
  res.json({
    totalSubmissions: submissions.length,
    totalFeedbacks: feedbacks.length,
    totalVotes: votes.length,
  });
});

app.listen(PORT, () => {
  console.log(`Analytics dashboard running on
http://localhost:${PORT}`);
});

```

Step 4: API Management and Expense Optimization

API Rate Limiting: Use the `rate-limit-axios` library to limit the number of API calls made to external services to manage costs effectively.

To manage your API calls effectively and minimize expenses, you can implement the following strategies:

1. Implement API Rate Limiting

Using a library like `rate-limit-axios`, you can control the number of API requests sent within a certain time frame. This is essential for staying within usage limits and avoiding unexpected charges.

Installation: Make sure you've already installed `axios` and `rate-limit-axios`:

```

bash

npm install rate-limit-axios

```

Code Example: Here's how to set up rate limiting for API calls in your bot:

```

javascript

const { RateLimiter } = require('rate-limit-axios');
const axios = require('axios');

// Set up rate limiter
const limiter = new RateLimiter({

```

```

    interval: 1000, // Time interval in milliseconds
    requests: 5,    // Maximum requests allowed in the interval
  });

// Example API call using the rate limiter
async function fetchData(url) {
  try {
    const response = await limiter.schedule(() => axios.get(url));
    return response.data;
  } catch (error) {
    console.error('API call error:', error);
    throw error;
  }
}

// Usage example
client.on('messageCreate', async (message) => {
  if (message.content.startsWith('!fetchData')) {
    const apiUrl = 'https://api.example.com/data'; // Replace with your API
    endpoint
    const data = await fetchData(apiUrl);
    message.channel.send(`Fetched data: ${JSON.stringify(data)}`);
  }
});

```

2. Centralized API Management

You can create a centralized API management system to handle all API calls efficiently. This involves setting up a service layer in your application that can manage and route requests.

Example Structure:

- **apiService.js:** This module handles all API interactions.

Code Example (apiService.js):

```

javascript

const axios = require('axios');
const { RateLimiter } = require('rate-limit-axios');

// Set up rate limiter
const limiter = new RateLimiter({
  interval: 1000,
  requests: 5,
});

// Fetch data from external API
async function fetchData(url) {
  try {
    const response = await limiter.schedule(() => axios.get(url));
    return response.data;
  } catch (error) {

```



```

        console.error('API call error:', error);
        throw error;
    }
}

// Export functions
module.exports = {
    fetchData,
};

```

Usage in Your Bot:

```

javascript

const apiService = require('./apiService');

client.on('messageCreate', async (message) => {
    if (message.content.startsWith('!getAPIData')) {
        const apiUrl = 'https://api.example.com/data'; // Replace with your API
        endpoint
        const data = await apiService.fetchData(apiUrl);
        message.channel.send(`Data from API: ${JSON.stringify(data)}`);
    }
});

```

3. Monitor API Usage

To further optimize costs, you can implement a monitoring system that logs API usage. This could involve tracking how many requests each function makes and setting alerts for when limits are approached.

Code Example: Here's a simple logging mechanism:

```

javascript

let apiRequestCount = 0;

async function fetchData(url) {
    apiRequestCount++; // Increment request count
    console.log(`API Requests made: ${apiRequestCount}`);

    try {
        const response = await limiter.schedule(() => axios.get(url));
        return response.data;
    } catch (error) {
        console.error('API call error:', error);
        throw error;
    }
}

// Reset count every hour (you can adjust this as needed)
setInterval(() => {
    apiRequestCount = 0;
}, 60 * 60 * 1000);

```

```
}, 3600000);
```

4. Batch API Requests

If the API you are using supports batch requests, consider grouping multiple requests into one to minimize the number of API calls. This can significantly reduce costs and improve performance.

Example:

javascript

```
async function fetchBatchData(urls) {
  const results = [];

  for (const url of urls) {
    const data = await fetchData(url);
    results.push(data);
  }

  return results;
}

// Usage example
client.on('messageCreate', async (message) => {
  if (message.content.startsWith('!fetchBatch')) {
    const urls = [
      'https://api.example.com/data1',
      'https://api.example.com/data2',
      // Add more URLs as needed
    ];

    const data = await fetchBatchData(urls);
    message.channel.send(`Fetched batch data: ${JSON.stringify(data)}`);
  }
});
```

5. Cache API Responses

To avoid making repeated requests for the same data, implement a caching mechanism. You can use in-memory caching or a simple database to store responses temporarily.

Example (using a simple object for in-memory caching):

javascript

```
const cache = {};

async function fetchData(url) {
  if (cache[url]) {
    console.log('Returning cached data for:', url);
    return cache[url];
  }
}
```

```
}

try {
  const response = await limiter.schedule(() => axios.get(url));
  cache[url] = response.data; // Cache the response
  return response.data;
} catch (error) {
  console.error('API call error:', error);
  throw error;
}
}
```

Summary:

With these strategies, you can build a robust and cost-efficient system for your Global AI Olympiad on Discord. Implementing API rate limiting, centralized management, usage monitoring, batch requests, and caching will help optimize expenses while ensuring smooth operation.

By carefully planning your API interactions and leveraging these techniques, you can create a highly efficient and scalable solution for your event.

Using the Bots on Discord

Overview of Bots

The following bots are available on the Discord server to assist you throughout the competition:

****1. Bot Name: AI Assistant Bot**

- **Purpose:** To provide answers to your AI-related questions and guide you in using AI tools.
- **Usage:** Type `!ask [your question]` in any channel to get instant answers. For example:
`!ask What are the best AI tools for data analysis?`
- **When to Use:** Whenever you have questions about AI concepts, tools, or best practices.

2. Project Management Bot

- **Purpose:** To help teams manage tasks and deadlines effectively.
- **Usage:** Use commands like `!addTask [task description]` to create tasks, or `!viewTasks` to see all tasks.
- **When to Use:** To keep track of project milestones, tasks, and responsibilities among team members.

3. API Integration Bot

- **Purpose:** To assist in making API calls for your AI project and fetch data.
- **Usage:** Type `!fetchData [API endpoint]` to retrieve data from the specified API.
- **When to Use:** When your project requires external data and you need to test API integrations.

4. Feedback Bot

- **Purpose:** To gather feedback from team members and provide insights on project progress.
- **Usage:** Use `!giveFeedback [your feedback]` to share your thoughts or `!viewFeedback` to see all feedback submitted.
- **When to Use:** At any stage of the project to encourage open communication and improvements.

5. Document Sharing Bot

- **Purpose:** To upload and share documents, PDFs, and other resources within your team.
- **Usage:** Type `!upload [file]` to upload documents for your team.
- **When to Use:** Whenever you need to share important resources or documents related to your project.

6. Analytics Bot

- **Purpose:** To track your project's progress and provide analytics on team performance.
- **Usage:** Type `!analytics` to get a summary of your project's performance metrics.
- **When to Use:** Periodically throughout the project to assess your team's progress and make necessary adjustments.

7. Event Reminder Bot

- **Purpose:** To send reminders about important deadlines and events.
- **Usage:** Type `!setReminder [date/time] [event description]` to create a reminder.
- **When to Use:** To ensure you and your team are aware of upcoming deadlines and the final event.

Best Practices for Using Bots

- **Explore Commands:** Familiarize yourself with the commands available for each bot. Use `!help` to get a list of commands.
- **Communicate Regularly:** Use the feedback and project management bots to keep communication open and transparent among team members.
- **Utilize Documentation:** If you have resources (like PDFs or links), use the document sharing bot to keep everything organized in one place.

- **Monitor Progress:** Regularly check the analytics bot to understand your team's performance and adjust strategies as needed.
-

Building Your Own Email Automation Bot

Creating your own email automation bot to manage outreach, follow-ups, and gather insights can be a great way to tailor the process to your specific needs. Below is a high-level guide on how to build and implement a custom email automation bot using a programming language like Python, along with various libraries and APIs.

Step 1: Define Requirements

- **Functionality:**
 - Send initial outreach emails.
 - Automate follow-up emails based on user responses or interactions.
 - Track email open rates and click-through rates.
 - Integrate with a central analytics dashboard.

Step 2: Choose Your Technology Stack

- **Programming Language:** Python is a good choice due to its robust libraries for email and API integration.
- **Email Sending Service:** Use SMTP servers (like Gmail) or services like SendGrid or Mailgun to send emails.
- **Database:** Use a simple database (like SQLite or PostgreSQL) to store user data, email interactions, and analytics.
- **APIs:** Consider using APIs for email tracking (like SendGrid) and analytics (like Google Analytics).

Step 3: Set Up Your Development Environment

1. Install Required Libraries:

```
bash

pip install smtplib email requests sqlite3
```

2. Create Project Structure:

```
arduino
email_bot/
├── bot.py
```

```
|— database.py
|— email_template.py
|— config.py
```

Step 4: Create a Database for Storing Data

- **database.py:**

```
python

import sqlite3

def create_connection():
    conn = sqlite3.connect('email_bot.db')
    return conn

def create_table(conn):
    cursor = conn.cursor()
    cursor.execute('''CREATE TABLE IF NOT EXISTS emails
                      (id INTEGER PRIMARY KEY,
                       email TEXT,
                       status TEXT,
                       opened BOOLEAN,
                       clicked BOOLEAN)''')

    conn.commit()

# Initialize database
conn = create_connection()
create_table(conn)
conn.close()
```

Step 5: Create Email Sending Functionality

- **bot.py:**

```
python

import smtplib
from email.mime.text import MIMEText
from email.mime.multipart import MIMEMultipart
from database import create_connection

def send_email(to_email, subject, body):
    # Email configuration
    sender_email = 'your_email@example.com'
    password = 'your_email_password'

    msg = MIMEMultipart()
    msg['From'] = sender_email
    msg['To'] = to_email
    msg['Subject'] = subject

    msg.attach(MIMEText(body, 'plain'))
```

```

try:
    server = smtplib.SMTP('smtp.example.com', 587)
    server.starttls()
    server.login(sender_email, password)
    server.send_message(msg)
    server.quit()

    print(f'Email sent to {to_email}')
    # Record email status in the database
    conn = create_connection()
    cursor = conn.cursor()
    cursor.execute("INSERT INTO emails (email, status, opened, clicked)
VALUES (?, ?, ?, ?)",
                  (to_email, 'sent', False, False))
    conn.commit()
    conn.close()

except Exception as e:
    print(f'Failed to send email: {e}')

```

Step 6: Automate Follow-Ups

- Add logic in your bot to check email statuses and send follow-ups if an email hasn't been opened after a certain period.
- **Example Logic:**

```

python

def follow_up():
    conn = create_connection()
    cursor = conn.cursor()
    cursor.execute("SELECT email FROM emails WHERE opened = False")
    emails_to_follow_up = cursor.fetchall()

    for email in emails_to_follow_up:
        send_email(email[0], "Follow-Up: Global AI Olympiad", "Don't miss the
chance to participate!")
    conn.close()

```

Step 7: Track Email Opens and Clicks

- Use a tracking pixel for open rates by embedding a small image in the email body that points to your server.
- Use unique links for clicks that redirect to your analytics server.
- Update the database when the pixel is accessed or a link is clicked.

Step 8: Central Analytics Dashboard

- Use libraries like Matplotlib or Seaborn to create visualizations based on the data collected in your SQLite database.
- Create a simple web app using Flask to display the analytics.

Step 9: Deploy Your Bot

- Host your bot on a cloud platform (like AWS, Heroku, or DigitalOcean) to run it continuously.
- Schedule the bot to run daily using a cron job or a task scheduler.

Example Code for Tracking Opens

Insert this in your email body:

```
html


```

Example Code for Tracking Clicks

Use unique links:

```
html

<a
href="https://yourdomain.com/track_click?email=recipient@example.com">Registe
r Now</a>
```

By building your own email automation bot, you can streamline the outreach process for your **Global AI Olympiad**, manage follow-ups, and gain valuable insights. This approach allows for greater customization, scalability, and control over your email marketing efforts.

! By leveraging Discord bots for tasks such as email automation, event management, and participant interaction, your team can focus more on media management and securing sponsors for the **Global AI Olympiad**. Here's a summary of how to structure your operations effectively:

Streamlined Task Management with Discord Bots

1. Task Automation

- **Email Automation Bot:**
 - Sends out initial outreach emails to university contacts and potential teams.
 - Handles follow-up emails based on responses.
 - Tracks open rates and engagement to refine future communications.

- **Event Management Bot:**
 - Manages event schedules, reminders, and deadlines.
 - Handles participant registrations and queries.
 - Provides updates and notifications about the competition stages.
- **Analytics Bot:**
 - Gathers data from the email bot and event bot.
 - Provides insights on engagement and participation rates.
 - Generates reports that can be shared with the team.

2. Media Management

- **Content Creation:**
 - Focus on producing promotional materials, social media posts, and video content.
 - Use the earlier suggested promotional pitch and video ad ideas to attract attention.
- **Social Media Engagement:**
 - Regular updates about the event on platforms like Twitter, LinkedIn, and Instagram.
 - Engage with potential participants, sponsors, and influencers to build buzz around the event.

3. Sponsorship Acquisition

- **Identifying Potential Sponsors:**
 - Create a list of companies that align with your event's mission (e.g., tech firms, educational institutions, financial organizations).
 - Reach out through personalized emails and proposals that highlight the benefits of sponsoring the event.
- **Sponsorship Packages:**
 - Develop different levels of sponsorship (e.g., Gold, Silver, Bronze) with corresponding benefits, such as branding opportunities, speaking slots, and promotional content.
- **Follow-Up Strategy:**
 - Use your email automation bot to manage follow-ups with potential sponsors.
 - Track interactions to determine interest and tailor follow-up messages accordingly.

Key Next Steps

1. **Finalize Bot Development:** Ensure all bots are fully functional and tested within your Discord server.
2. **Content Calendar:** Create a content calendar for social media and media outreach to keep your marketing efforts organized.
3. **Sponsorship Outreach:** Start drafting sponsorship proposals and reach out to potential sponsors to gauge interest.

4. **Team Meetings:** Regularly check in with your team on Discord to monitor progress and adjust strategies as needed.

By streamlining these processes, you can ensure that your team remains focused on high-impact tasks while the bots handle the repetitive, administrative work. This will not only improve efficiency but also enhance your ability to engage with participants and sponsors effectively

Centralized Data Management System

To effectively gather and manage the data collected by your Discord bots, you can set up a centralized data storage and management system. Here's how you can structure this system, including where to store the data, how to access it, and how to analyze it:

1. Data Storage

You can use a database to store all the information collected by the bots. Here are some options for setting up your database:

- **Database Options:**
 - **SQLite:** A lightweight database that's easy to set up for smaller projects. Ideal for initial development.
 - **PostgreSQL:** A more robust option that supports complex queries and larger datasets. Great for scalability.
 - **MySQL:** Another widely-used relational database that works well for web applications.

Recommended Database Structure:

- **Participants Table:**
 - **id:** Unique identifier
 - **name:** Participant name
 - **email:** Participant email
 - **team_id:** Associated team (if applicable)
 - **registration_status:** Status of registration (pending, confirmed)
- **Emails Table:**
 - **id:** Unique identifier
 - **email:** Email address
 - **subject:** Subject of the email
 - **status:** Status (sent, opened, clicked)
 - **created_at:** Timestamp of when the email was sent
- **Sponsors Table:**
 - **id:** Unique identifier

- company_name: Name of the sponsoring company
- contact_person: Person of contact
- email: Contact email
- status: Sponsorship status (interested, confirmed, declined)
- **Event Analytics Table:**
 - id: Unique identifier
 - event_name: Name of the event
 - participants_count: Number of participants registered
 - email_open_rate: Open rate of emails
 - engagement_rate: Engagement metrics (e.g., responses, interactions)

2. Accessing Data

- **Data Collection:**
 - Each bot should be programmed to collect relevant data and store it in the database after each interaction.
 - For example, the email bot should log each email sent and track whether it was opened or clicked.
- **Data Retrieval:**
 - Create a simple API or use a web framework like Flask or Django to set up endpoints to retrieve data from your database.
 - Use these endpoints to query the database for insights or to generate reports.

Example API Endpoint:

```
python

from flask import Flask, jsonify
from database import create_connection

app = Flask(__name__)

@app.route('/api/participants', methods=['GET'])
def get_participants():
    conn = create_connection()
    cursor = conn.cursor()
    cursor.execute("SELECT * FROM participants")
    participants = cursor.fetchall()
    conn.close()
    return jsonify(participants)

if __name__ == '__main__':
    app.run(debug=True)
```

3. Data Analysis

- **Analytics Tools:**
 - Use libraries like **Pandas** or **Matplotlib** in Python to analyze and visualize the data.
 - Create scripts that run periodically to generate reports on participant engagement, email success rates, and sponsorship interest.
- **Dashboard:**
 - Set up a dashboard using tools like **Tableau**, **Grafana**, or **Google Data Studio** to visualize key metrics and track progress over time.
 - This dashboard can pull data from your API or directly from your database.

Summary

1. **Set up a database** to store all the collected data from the Discord bots.
2. **Program the bots** to log their activities into this database after each interaction.
3. **Create APIs** to retrieve and manage data efficiently.
4. **Use data analysis tools** to gain insights and generate reports to inform your strategy moving forward.

By having a centralized data management system in place, you'll be able to keep track of all the important metrics, streamline communication, and make data-driven decisions for the **Global AI Olympiad**.

Revised Bot Structure

Instead of having eight separate bots, you will be developing **three core bots** for your Discord server that will cover the necessary functionalities. Additionally, you'll implement a **dashboard for analytics** to streamline data management and insights. Here's how you can structure it:

1. **Email Automation Bot**
 - **Functionality:**
 - Send outreach emails to universities for team recruitment.
 - Manage follow-up emails and track engagement (opens, clicks).
 - Log interactions and insights into the database.
 - **Training Capability:** Can be trained on data, PDFs, and links to improve email content and targeting.
2. **Event Management Bot**
 - **Functionality:**
 - Handle participant registrations and manage queries.
 - Send notifications for key events and deadlines.
 - Log registration data and participant engagement into the database.
3. **Analytics Bot**
 - **Functionality:**

- Collect and analyze data from the Email Automation Bot and Event Management Bot.
- Generate reports on participant metrics, email performance, and overall engagement.
- Integrate with a dashboard to visualize key insights in real-time.

Dashboard for Analytics

- **Centralized Dashboard:**
 - A single dashboard will be set up to visualize key metrics and data collected by the bots.
 - This can include:
 - **Participant Analytics:** Registration numbers, demographics, and engagement levels.
 - **Email Performance Metrics:** Open rates, click-through rates, and responses.
 - **Sponsorship Insights:** Status of sponsorship acquisition and interest levels.

Benefits of This Structure

- **Streamlined Management:** Fewer bots simplify maintenance and integration.
- **Focused Functionality:** Each bot has a clear purpose, making it easier to develop and train.
- **Centralized Data Insights:** The dashboard allows for real-time monitoring and analysis, which can inform your strategy and decision-making.
-

Next Steps

1. **Develop Each Bot:** Assign your tech team to build out the functionalities of each bot as specified.
2. **Create the Dashboard:** Set up the dashboard using tools like Tableau, Grafana, or Google Data Studio to visualize the data collected by the bots.
3. **Integrate Bots with Database:** Ensure that each bot is correctly logging its activities and insights into your centralized database.
4. **Test Functionality:** Conduct testing to ensure that all bots are working as intended and that data flows seamlessly to the dashboard.

By focusing on these three bots and a centralized dashboard, you will have an efficient system that can effectively manage the event while providing you with valuable insights to optimize your strategy and execution.

Step-by-Step Guide for Creating and Integrating Discord Bots

Here's a high-level step-by-step guide for creating and integrating the **Email Automation Bot**, **Event Management Bot**, and **Analytics Bot** into your Discord server. This guide assumes you have some familiarity with programming (Python is commonly used for Discord bots) and access to a Discord Developer account.

Step 1: Set Up Your Development Environment

1. Install Required Tools:

- Install Python (version 3.8 or higher).
- Install `discord.py` library:

```
bash
```

```
pip install discord.py
```

- Install any additional libraries you might need for email automation (like `smtplib` or `email`) and data handling (like `pandas`).

2. Set Up Your Project Structure:

- Create a project folder (e.g., `discord_bots`).
- Inside this folder, create three separate files for your bots (e.g., `email_bot.py`, `event_bot.py`, `analytics_bot.py`).

Step 2: Create Your Discord Bot Applications

1. Go to the Discord Developer Portal:

- Visit Discord Developer Portal.

2. Create a New Application:

- Click on "New Application."
- Name your application (e.g., "Global AIOlympiad Bots").

3. Create Individual Bots:

- Under the application settings, navigate to the "Bot" tab.
- Click on "Add Bot" to create your bot for each of the three functionalities.
- Note down the **Bot Token** for each bot (you will use this in your code).

4. Set Permissions:

- Under the "OAuth2" tab, set the required permissions for your bots, such as sending messages, managing messages, etc.
- Generate an invite link to add the bot to your server.

Step 3: Code Each Bot

1. Email Automation Bot (`email_bot.py`):

- Import required libraries:

```
python
```



```
import discord
import smtplib # or any email library you plan to use
```

- Create a class for the bot and implement email functionalities (sending emails, tracking responses).
- Example skeleton:

```
python

class EmailBot(discord.Client):
    async def on_ready(self):
        print(f'Logged in as {self.user}')

        # Define email functionalities (send, log, track)
```

2. Event Management Bot (`event_bot.py`):

- Import the discord library.
- Create a class for the event management functionalities (registration handling, notifications).
- Example skeleton:

```
python

class EventBot(discord.Client):
    async def on_ready(self):
        print(f'Logged in as {self.user}')

        # Define functionalities for handling registrations and events
```

3. Analytics Bot (`analytics_bot.py`):

- Import libraries for data collection and reporting.
- Create a class for the analytics functionalities (collect data from other bots and generate reports).
- Example skeleton:

```
python

class AnalyticsBot(discord.Client):
    async def on_ready(self):
        print(f'Logged in as {self.user}')

        # Define analytics functionalities
```

Step 4: Integrate Bots with Discord Server

1. Run Each Bot:

- Ensure each bot has a main loop to run:

```
python
```

```

if __name__ == "__main__":
    email_bot = EmailBot()
    email_bot.run('YOUR_EMAIL_BOT_TOKEN')

    event_bot = EventBot()
    event_bot.run('YOUR_EVENT_BOT_TOKEN')

    analytics_bot = AnalyticsBot()
    analytics_bot.run('YOUR_ANALYTICS_BOT_TOKEN')

```

2. Test Bots in Discord:

- Invite the bots to your Discord server using the invite link generated earlier.
- Test functionalities in a designated channel to ensure everything works as expected (sending messages, handling registrations, etc.).

Step 5: Set Up a Centralized Database

1. **Choose a Database:** Select a database management system (e.g., PostgreSQL, MySQL).
2. **Create Tables:** Define the necessary tables for logging bot activities, participants, and analytics.
3. **Integrate Database with Bots:** In each bot's code, connect to the database and implement methods for logging relevant data:
 - Use libraries like `SQLAlchemy` for easy database integration.

Step 6: Implement the Dashboard

1. **Choose a Dashboard Tool:** Use tools like Tableau, Grafana, or Google Data Studio for visualization.
2. **Connect to Database:** Ensure your dashboard connects to the same database for real-time data access.
3. **Define Metrics:** Set up the metrics you want to display based on the data collected from the bots.

Step 7: Continuous Testing and Improvement

1. **Conduct Regular Tests:** Regularly test each bot to ensure they are functioning properly and collecting data correctly.
2. **Gather Feedback:** After initial deployment, gather feedback from users to improve bot functionalities and user experience.
3. **Update Bots:** Regularly update the bots based on feedback and new requirements.

This guide provides a high-level overview of the steps needed to create and integrate your three Discord bots. As you progress, you may need to dive deeper into specific coding techniques or tools, but this framework will help you get started.

Step-by-Step Guide to Set Up a Database for Your Bots

Step 1: Choose a Database

For your project, you can choose a relational database like **PostgreSQL** or **MySQL**, or a NoSQL database like **MongoDB**. For simplicity, we'll use **SQLite** in this example, as it's easy to set up and doesn't require a separate server.

Step 2: Install Required Libraries

Make sure to install the necessary libraries for database interaction. If you're using SQLite, you can use the built-in `sqlite3` module. If you choose PostgreSQL or MySQL, install `SQLAlchemy` and the respective driver:

```
bash
```

```
pip install sqlalchemy sqlite3 # For SQLite
pip install sqlalchemy psycopg2 # For PostgreSQL
pip install sqlalchemy mysqlclient # For MySQL
```

Step 3: Create a Database

You can create a database file if you are using SQLite. For PostgreSQL or MySQL, you need to create a database on the server.

For SQLite:

```
python
```

```
import sqlite3

# Connect to the SQLite database (or create it if it doesn't exist)
conn = sqlite3.connect('olympiad.db')

# Create a cursor object to execute SQL commands
cursor = conn.cursor()

# Create tables
cursor.execute('''
CREATE TABLE IF NOT EXISTS participants (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    email TEXT NOT NULL,
    registration_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP
)
''')
```

```

cursor.execute('''
CREATE TABLE IF NOT EXISTS emails (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    email TEXT NOT NULL,
    status TEXT NOT NULL,
    sent_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP
)
''')

cursor.execute('''
CREATE TABLE IF NOT EXISTS analytics (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    event TEXT NOT NULL,
    data TEXT NOT NULL,
    recorded_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP
)
''')

# Commit changes and close the connection
conn.commit()
conn.close()

```

Step 4: Integrate the Database with Your Bots

In each of your bot scripts, you will need to connect to the database and implement functions to insert and retrieve data.

Example for Email Automation Bot:

```

python

import sqlite3

def log_email(email, status):
    conn = sqlite3.connect('olympiad.db')
    cursor = conn.cursor()

    cursor.execute('''
INSERT INTO emails (email, status) VALUES (?, ?)
''', (email, status))

    conn.commit()
    conn.close()

def send_email(email):
    # Your email sending logic here
    # After sending the email, log the email
    log_email(email, 'Sent')

```

Example for Event Management Bot:

```
python
```

```
import sqlite3

def register_participant(name, email):
    conn = sqlite3.connect('olympiad.db')
    cursor = conn.cursor()

    cursor.execute('''
INSERT INTO participants (name, email) VALUES (?, ?)
''', (name, email))

    conn.commit()
    conn.close()
```

Example for Analytics Bot:

python

```
import sqlite3

def log_event(event, data):
    conn = sqlite3.connect('olympiad.db')
    cursor = conn.cursor()

    cursor.execute('''
INSERT INTO analytics (event, data) VALUES (?, ?)
''', (event, data))

    conn.commit()
    conn.close()

def generate_report():
    conn = sqlite3.connect('olympiad.db')
    cursor = conn.cursor()

    cursor.execute('SELECT * FROM analytics')
    reports = cursor.fetchall()

    conn.close()
    return reports
```

Step 5: Testing the Database Integration

1. **Run Your Bots:** Start each bot and perform actions that trigger database interactions (e.g., register participants, send emails).
2. **Check the Database:** Use a tool like SQLite Browser or connect via the command line to check the contents of your database tables and verify that data is being logged correctly.

This setup provides a simple database structure to log participants, emails, and analytics data. You can expand the schema based on your requirements.

Immediate Action Items Summary

1. Outline the event structure and set key dates.
2. Develop and test the necessary Discord bots.
3. Set up the database and ensure proper connections.
4. Create sponsorship packages and begin outreach.
5. Develop a marketing plan and promotional materials.
6. Set up analytics and reporting systems.
7. Coordinate with your team and maintain communication.

Timeline Overview

- **October - November 2024:** Complete immediate tasks and finalize event structure.
- **December 2024:** Launch the Olympiad with registration open.
- **January - March 2025:** Run preliminary rounds and provide mentorship to teams.
- **April - May 2025:** Final preparations, including sponsor acquisition and event promotion.
- **June 2025:** Host the final gala event in London.

By focusing on these immediate tasks, we can create a strong foundation for our **Global AI Olympiad** and ensure everything is ready for a successful launch in December.

